SEASUG Paper ###-2026

The SAS Hash Object Beyond the Docs as Experimental Science

Paul M. Dorfman, Independent SAS Consultant

ABSTRACT

When one learns a new piece of SAS software, one's primary resource is the SAS documentation, usually augmented by SAS papers, books, online help and of course code cannibalization, all of which are helpful. However, sometimes the most important resource is the learner oneself willing to play with SAS code intelligently, especially one with a scientific background. This is particularly true of the SAS hash object since learning it via programming experiment can make many things the docs are vague about clear, reveal facts the docs are silent about, correct things the docs are wrong about, and lead to real hash object mastery without the need to rely on code cannibalization. This is the way the author has been able to learn the SAS object well beyond its docs since its advent in 2003 while sharing the findings, sometimes revelatory, with the SAS community. In this presentation, he hopes to intrigue the audience by demonstrating both his experimental setups and conclusions by coding, running, and examining what he preaches - live.

INTRODUCTION

At the beginning of 2003, I received an email message from Mr. Al Kulik from SAS. He presented himself as a Base R&D developer working on implementing new canned functionality based on hash algorithms. After citing my older SAS papers on bespoke array hashing as a precursor, he told me that the SAS Version 9.0, to be unveiled in 2003, had an experimental version of the associative array, aka the hash object. Mr. Kulik asked if I would like to install the SAS 9.0 beta on my Windows machine, peruse the draft of the hash object docs he had attached, “have at it”, and provide some feedback.

Needless to say, I dove into it headfirst, which was all the more interesting that the original docs were also kind of experimental. While there was enough to start poking around, it was basically a cheat sheet adorned with some extra verbiage. The docs had a lone, yet well-chosen use case: A code example of matching two data sets using the hash object as a lookup table. The rest, it seemed, had to be found by a programmatic experiment, so I started experimenting and discovering fascinating functionality in the process.

As the SAS hash object continued to evolve and its functionality kept expanding, so did its docs - and not without curious twists. For instance, certain keywords, while still fully functional today, were taken out from the docs, some vague or even erroneous descriptions were kept intact, and several important caveats were never addressed adequately. Most importantly, the docs never treated the hash object as a truly dynamic structure, missing out on its most fundamental property. Every such incongruity required more experimentation before the hash object could be used with due programming awareness and to its full potential.

In this paper, I would like to invite you to my little lab and look at some of my experimental setups, methods, and reasoning along with results and their interpretations. If you choose to attend the presentation, you will have more fun than just reading the paper by observing the actual SAS code experiments being demoed live.

DECLARE

The very first thing about the hash object was this:

```
declare hash H ;
```

The docs said that this is used to *declare a hash object* – and nothing else. Now the docs add that the statement tells SAS that the object reference H is a hash object. To a pre-hash SAS programmer, neither tells much. For example, “the object reference H is a hash object” is not quite clear: Is H a reference to the hash object H or is it the hash object itself? Also, the original docs said that DECLARE can be abbreviated as DCL – which, interestingly is omitted from the current docs. So, the first inclination was just to run the statement in both variants and see what happens, and that’s what I did:

```
data _null_ ;
  declare hash H ;
run ;

data _null_ ;
  dcl hash H ;
run ;
```

Both run with no errors or warnings or any other issues. However, the first question I wanted to find an answer to was:

Is DECLARE a Compile Time or Run Time Statement?

Since SAS declarative statements (e.g. LENGTH) are normally compile-time, that was my strong suspicion. To confirm it, I set up the following experiment: Declare two hash objects, H and X. Run the DCL statement for H *outside* a DO loop iterating 100,000 times; then run the one for X *inside* the same loop; then compare the run times. If the execution time for X is palpably longer than for H, then DCL is a run-time statement; otherwise, it is compile-time because running a compile-time statement inside a loop has no effect on the execution time.

```
data _null_ ;
  t = time() ;
  dcl hash h ;
  do i = 1 to 1e5 ;
  end ;
  tH = time() - t ;
  put tH= ;

  t = time() ;
  do i = 1 to 1e5 ;
    dcl hash x ;
  end ;
  tX = time() - t ;
  put tX= ;
run ;
```

The result for running this step is tH=0 and tX=0 printed in the log. So, we must conclude that the DCL statement is indeed a compile-time directive. With that off the table, the second thing to discover, more nagging to my geeky nature, was the true nature of H, i.e. what kind of SAS entity it is. Or borrowing a line from the Bard, we can ask about H:

What’s In a Name?

In an experiment, sometimes an incidental mistake can lead to a clue. In my case, I decided to verify if I could code “associativearray” instead of “hash” since that is what the original docs suggested. In fact,

they originally called the hash object exactly that, an associativearray (and indeed for good reason but that is beyond the scope here). So, I tested this:

```
data _null_ ;
  dcl hash H ;
  dcl associativearray H ;
run ;
```

And, in response to running this step, the log reported: `ERROR 567-185: Variable H already defined.` I realized that I had goofed, for if H is *any array* (associative or not), the compiler does not allow it to be declared twice. Changing the second H to, for example, X makes the error go away. However, it was an example of one of those lucky slips: For even though the original docs said nothing about it (and neither do the current docs), the error message text written by the *developer* (rather a tech writer) reveals the true nature of the language element H after the keyword HASH, to wit: ***H is actually a variable.***

Furthermore, it is obvious that the compiler stores it in its symbol table since the error above is evidently its reaction to the attempt of storing H in the symbol table after H is already there. Of course, I could not help but check if the same was true about the hash iterator object:

```
data _null_ ;
  dcl hash H ;
  dcl hiter HI ("H") ;
  dcl hiter HI ("H") ;
run ;
```

in which case the log reports the same error: `ERROR 567-185: Variable HI already defined`. Since now I knew that H (or HI) is a *variable*, it raised other questions. For example, since it is a variable, we should be able to assign a value to it, right? And then, since a variable is like a box with a value inside it, I was curious, not unlike Pandora, to see what was inside the box, so I tried:

```
data _null_ ;
  dcl hash H ;
  H = 1 ;
  put h= ;
run ;
```

Then I repeated the same, just changing 1 to "A" to test both numeric and character types. For each data type, this step generates the same two errors: one for line 3 and one – for line 4, respectively:

```
ERROR: Scalar cannot be converted to object of type hash.
ERROR: Invalid operation for object type.
```

Getting errors in the log might have an unpleasant feeling – but they are an invaluable source of learning, especially if the docs are scant or vague about what one wants to know. From these two errors, I learned two important things:

- H is not a scalar variable but rather a *variable of type hash*.
- A scalar value, such as the SAS numeric or character value, cannot be assigned to a non-scalar variable, such as a variable of type hash.

Then I got the same results by trying all the above for the hash iterator, for example:

```
data _null_ ;
  dcl hash H ;
```

```
  dcl hiter HI ("h") ;
  hi = 1 ;
  put hi= ;
run ;
```

Testing this step results in the same errors as with H, with the only distinction that "type hash" is replaced with "type hiter".

```
ERROR: Scalar cannot be converted to object of type hiter.
ERROR: Invalid operation for object type.
```

Okay, now that I knew what could not be done with variables like H or HI, it was logical to explore the more practical question of what *could* be done, i.e.: How to assign a value to a non-scalar variable like H after it has been declared? Running a tad bit ahead, I found *two* answers to this.

Is It a Must to Declare Separate Hash Objects Separately?

While still experimenting with the hash declaration, I at times needed to declare separate hash object reference variables, for example, H, H2, H3, etc. Naturally, I would code:

```
data _null_ ;
  dcl hash H  ;
  dcl hash H1 ;
  dcl hash H2 ;
run ;
```

Of course, that worked just fine. However, I thought that if I had many more objects to declare, it might get too verbose. On the other hand, DECLARE is, well, a declarative statement, and I knew that declarative statements in SAS usually allowed us to handle more variables than one in a single statement, for example:

```
length v v1 v2 $ 8 ;
```

So, it was natural to wonder whether this would work instead of the three separate statements above:

```
data _null_ ;
  dcl hash H H2 H3 ;
run ;
```

And guess what? It did work! – proven by the fact that the step above rran error-free, which in turn meant that the DATA step compiler accepted the syntax. Now, whether the hash objects declared in this concise manner was functionally identical to the ones declared in the separate statements, remained to be tested separately – more on that later.

INSTANCE CREATION AND IDENTIFICATION

The first technique of assigning a non-scalar value to H was more or less obvious from the docs in the form of the statement following the DCL statement:

```
data _null_ ;
  dcl hash H ;
  h = _new_ hash() ;
run ;
```

Indeed, when a line of SAS code starts with the construct <*variable name*> =, it sure looks like an assignment statement, does it not? The docs said that the _new_ operator creates an instance of object, whose reference is H. However, they kept mum about what kind of thing is *actually* assigned to the variable H. I found the answer later via a more elaborate experiment. More on that later; at that point, though, I decided to test whether the statement in bold above is compile-time or run-time. I strongly suspected the latter but decided to test it this way:

```
option fullstimer ;
 * step 1 ;
data _null_ ;
  dcl hash H ;
  h = _new_ hash() ;
  do i = 1 to 10000 ;
  end ;
run ;
* step 2 ;
data _null_ ;
  dcl hash H ;
  do i = 1 to 10000 ;
    h = _new_ hash() ;
  end ;
run ;
```

Step 1 ran in 0.01 seconds, while step 2 took 0.14 seconds, i.e. 140 times longer. It certainly proves that this is a run-time statement, for one would expect that executing a run-time instruction many times takes more time than doing it once. However, the comparison of their memory footprints was really puzzling: Step 1 used 354.25k, while step 2 used a whopping 143450.75k. Why?! In the pre-hash era, the loop above merely overwrites the content of H in each iteration, so it should not affect the memory footprint. If the _NEW_ operator likewise simply assigned a new value to the same variable, there would be no difference. Hence, the only logical hypothesis to explain that was:

- Each time the DO loop iterates, the _NEW_ operator creates a new *physical* instance of the hash object with its own memory footprint.
- 10000 such instances add up to take up 10000 times more RAM than a single instance.

This experiment cannot answer the question: Does the _NEW_ operator create a new value to assign to H to identify the new instance it creates? A separate experiment is needed to elucidate that. However, if it does, it still overwrites the value in H with every new value. Judging from the memory footprint, all the instances created in the loop *physically persist* and take up memory, yet all the values identifying them in H are overwritten except the last one.

That begs another question – and a good one, too: Can we somehow reference the rest of the 9999 instances created – or do they remain useless garbage just polluting the memory? Unfortunately, because none of them, before having been overwritten in H, had been stored elsewhere outside H, there is no way to retrieve their references. Hence, if we create many hash object instances, say H, H1, H2, … and want to reference them distinctly, there are two possibilities:

1. Create only one instance of H, H1, H2 and reference them directly by their names, e.g.: h.definekey(), h1.add(), h2.find().
2. Create three instances of H in three separate identical statements – or in a loop - but save them somewhere in some specific way – *more on that later*.

Declare and Create an Instance – Single Statement

Instead of coding the DCL statement and the assignment statement with the _NEW_ operator separately, the docs said it was possible to combine them into one:

```
data _null_ ;
  * dcl hash H ;
  * h = _new_ hash() ;
  dcl hash H() ;
run ;
```

This single-statement approach is interesting because it combines a purely compile-time DCL statement and a purely run-time statement using the _NEW_ operator. I ran them through the following double-step experiment to find how the double-statement and single-statement declaration/initialization compared with one another:

```
data _null_ ;
  dcl hash H ;
  do i = 1 to 10000 ;
    h = _new_ hash() ;
  end ;
run ;

data _null_ ;
  do i = 1 to 10000 ;
    dcl hash H() ;
  end ;
run ;
```

Running it shows virtually no differences in either execution time or memory usage. This outcome is logical: In the DO loop, the compile-time statement is ignored at run time, and only the run-time part contributes to the execution time. So, for all intents and purposes the two-statement and single-statement techniques are identical.

And yet, there is one exception! With the separate-statements approach, we can create, say, two different instances of H using either of the following:

```
data _null_ ;
  dcl hash H ;
  do i = 1 to 2 ;
    h = _new_ hash() ;
  end ;
run ;

data _null_ ;
  dcl hash H ;
  h = _new_ hash() ;
  h = _new_ hash() ;
run ;
```

Both ran successfully – no errors, no warnings. And both do the same things: (1) create one instance of H and assign its non-scalar ID value – say, ID1 - to H and (2) create another instance of H and assign its non-scalar ID value – say, ID2 - to H, overwriting ID2 in there. However, from what I already knew, I had expected that the following would fail because the compile-time parts of each statement would be in conflict:

```
data _null_ ;
```

```
  dcl hash H() ;
  dcl hash H() ;
run ;
```

And indeed, running this step resulted in our old nemesis at line #3: `ERROR 567-185: Variable H already defined` – apparently, because the compile-time part of DCL had already created the variable H at line #2. The conclusion is that to generate multiple instances of the same hash object H using the single-statement declare/instantiate method, a loop is required. It does not have to be necessarily a DO loop – the DATA step implied loop will do it, too. As an exercise, run the two steps below and compare the results both run-time and memory-wise:

```
* Step 1 ;
data _null_ ;
  set sashelp.zipcode (drop=_all_) ;
run ;
* Step 2 ;
data _null_ ;
  dcl hash H() ;
  set sashelp.zipcode (drop=_all_) ;
run ;
```

Since the file has sashelp.zipcode has 41232 observations, the implied loop executes the DCL statement's run-time part 41232 times and creates a new persistent instance of H at every iteration. Correspondingly, the memory footprints and run times of step 1 vs step 2 are 661k vs 580928k and 0.02 sec vs 0.42 sec. The explains why the docs recommend that run-time statement intended to create a single instance of a hash object is placed inside the IF _N_ = 1 THEN… construct, as in:

```
data _null_ ;
  if _n_ = 1 then do ;
    dcl hash H() ;
  end ;
  set sashelp.zipcode (drop=_all_) ;
run ;
```

With this change, the stats of step 2 become almost identical to step 1.

Last but not least for this section, one could not help but wonder if a DCL statement that declares multiple hashes at once could include, in addition to purely declarative H, H1, H2, etc., also *combined declarative and instantiating* code like H2() in the step below:

```
data _null_ ;
  dcl hash H H1 H2() ;
run ;
```

And, to my delight, I discovered that this step runs error-free! Above, the hashes H and H1 are *merely declared*, whereas H2 is *both declared and instantiated*. In other words, after DCL is executed, the variable of type hash H2 contains the ID-value of its instance, whereas H and H1 do not. We will see the difference it makes in the following section.

Hash ID Assignment Gymnastics

In the step above, all hash-ID variables H, H1, H2 are of the same type – type hash. Therefore, I had expected that there would be no problem with any of following assignments:

```
data _null_ ;
```

```
  dcl hash H H1 H2() ;
  H = H1 ;
  H = H2 ;
run ;
```

However, contrary to expectations, the step abends with the following log messages:

```
NOTE: Variable H1 is uninitialized.
ERROR: Object type mismatch at line 3 column 3.
ERROR: Assigned object must be of type hash.
```

Interestingly, there is no problem with assigning H2 to H, and once H1 is changed to H1(), the note and error messages disappear. The error messages in this case look kind of confusing; but I cannot help but conclude that the problem occurs only when H1 is not instantiated. The latter means that the variable H1 at the time of assignment has no value. That is not surprising since no instance of H1 has been created, and so no non-scalar value that would identify the instance has been stored in H1. From this, I concluded that a hash-ID variable H could be assigned the ID value of another hash-ID variable H1 only if H1 were populated with a non-missing hash-ID value. However, as we have just seen, the *receptacle* hash-ID variable, such as H above, does not have to be instantiated to receive an ID value from another hash-ID variable.

Let us consider a situation where understanding of such intricacies may be valuable. Suppose we have two instantiated, populated, and initialized hash objects H1 and H2:

```
data _null_ ;
  dcl hash H H1() H2() ;
  h1.definekey ("k") ;
  h1.definedone () ;
  h2.definekey ("k") ;
  h2.definedone () ;
  do K = 1 to 7 ;
    h1.add() ;
    if 3 <= K <= 5 then h2.add() ;
  end ;
run ;
```

This step runs flawlessly. Let us call the code inside it “existing code”. Now suppose we want to do 3 identical things with both H1 and H2:

- Find the number of items in each instance.
- Find if the key K = 5 is in each.
- Clear both H1 and H2 .

Correspondingly, we could add the following after <existing code>:

```
data _null_ ;
  <...existing code...>
  n_items  = h1.num_items ;
  rc_check = h1.check (key:5) ;
  rc_clear = h1.clear() ;
  put "hash=1 " (n_items rc_check rc_clear) (=) ;

  n_items  = h2.num_items ;
  rc_check = h2.check (key:5) ;
  rc_clear = h2.clear() ;
```

```
    put "hash=2 " (n_items rc_check rc_clear) (=) ;
  run ;
```

That runs flawlessly, too, and prints in the SAS log:

hash=1 n_items=7 rc_check=0 rc_clear=0
hash=2 n_items=3 rc_check=0 rc_clear=0

One problem with the above approach is that it is verbose – and would be more so if we had more than only two hash objects to process in the similar manner. Notice, though, that in the "existing code", we have defined the hash-ID variable H - on purpose, of course. Now let us see how we can utilize it to call only one version of the same code for both H1 and H2:

```
  data _null_ ;
    <...existing code...>
    h = h1 ; hash = 1 ; link process ;
    h = h2 ; hash = 2 ; link process ;
    stop ;
    process: n_items  = h.num_items ;
             rc_check = h.check (key:5) ;
             rc_clear = h.clear() ;
             put (hash n_items rc_check rc_clear) (=) ;
    return ;
  run ;
```

This runs correctly, too, and produces the same results - *but how*? Note that in the PROCESS link routine, there are *no references to H1 or H2 but only references to H*. So, how does the program know when to apply the link routine code to H1 or H2? Well, the results of this experiment look convincing enough to propose the following theory.

Suppose we have an instance of the hash object references by a hash-ID variable X generated by either:

```
 dcl hash H ;
  h = _new_ hash h() ;
```

or:

```
 dcl hash H() ;
```

Executing either variant makes the program:

- Generate a unique non-scalar value of type hash to serve as this instance's ID-value. We cannot display or see that value; but let us *imagine* that the ID-value is 7.
- Tag the physical hash instance with this ID-value = 7.
- Assign this ID-value = 7 to the hash-ID variable H.

When any hash attribute or method references H, such as h.num_items, h.check(), etc., the following occurs:

- The program accesses the hash-ID variable H and retrieves the ID-value stored in H, which is 7.
- It uses this ID-value = 7 as the key to locate the physical hash instance tagged with ID-value = 7.
- From that point on, any hash tool dot-referenced by H is going to work on the hash instance tagged with the ID-value = 7.

Suppose we already have another hash object instance created before and tagged with the generated ID_value = 3, also stored in the hash-ID variable H1. Now let us see what happens if we code:

```
H = H1 ;
```

After this assignment is executed, the value stored in H1 is assigned to H, so now H contains the ID_value = 3. Therefore, now when any hash attribute or method references H, such as h.num_items, h.check(), etc.:

- The program accesses the hash-ID variable H and retrieves the ID-value stored in H. However, now it is 3, not 7.
- It uses this ID-value = 3 as the key to locate the physical hash instance tagged with ID-value = 3.
- From that point on, any hash tool dot-referenced by H is going to work on the hash instance tagged with the ID-value = 3.

Therefore, by overwriting the ID-value store in the hash-ID variable H with ID-values of other existing hash instances, we can make the program work on different hash tables by using a single hash-ID variable reference – in this case, H. From the standpoint of this theory, the mechanics of the program above can be described as follows:

- When the _NEW_ operator or the fused DCL statement creates an instance of H1, it creates an instance of H2, the variables H1 and H2 get populated with non-scalar ID values.
- There are no means to "see" these values, but they are real and there are places for them in the PDV.
- Let us, as a matter of convenience, *call* the ID value stored in H1 "ID1" and the ID value stored in H2 "ID2".
- Also, at the time of instantiation, the physical instances of H1 and H2 are tagged as "ID1" and "ID2".
- When we refer to H in the object-dot notation (e.g. h.num_items, etc.), the ID value stored in H tells the respective hash tool to work on the physical instance tagged with that value.
- When we assign H = H1, we copy "ID1" from H1 to H, and since now H contains "ID1", any hash tool referenced by H knows to work on the instance tagged by "ID1".
- When we assign H = H2, we copy "ID2" from H2 to H, and since now H contains "ID2", any hash tool referenced by H knows to work on the instance tagged by "ID2".
- That is why we can switch between making the program work on the instance H1 or instance H2 by merely changing the value of H and referring to every hash tool involved to H instead.

As a side note, understanding this mechanism is critical to comprehending how the so-called *hash-of-hashes (HOH) technique* actually works.

SMALL EXPERIMENT – BIG LESSON

Sometimes the simplest and easiest "curiosity" experiment leads to profound expansion of knowledge. The "expressions" experiment described below is one of those.

Does It Have to Be a Literal?

Practically every example of coding in the hash docs contains SAS *constant literals*. In the SAS language, literals are code elements representing constant hard-coded values, such as 1, "ABC", etc. Emulating (not verbatim) an example from the docs, we can code something like this:

```
data _null_ ;
  length K D 8 ;
  dcl hash h (hashexp:16, ordered: "a") ;
  h.definekey  ( "k" ) ;
  h.definedata ( "k" , "d" ) ;
```

```
    h.definedone () ;
    k = 1 ; d = 2 ; h.add() ;
  run ;
```

In the code above, the literals are shown in **boldface**. Nothing in the docs suggests that these elements – in this case, method arguments and argument tags - can be anything but hard-coded literals. However, in the pre-hash-object era, every SAS programmer knew that literals are merely the simplest case of SAS *expressions*. In their hierarchy, literals are the simplest expressions; the next higher up are *variables*. So, it was natural for me to wonder, from the outset, what would happen if I replaced the literals with valued variables – of the same data type, of course. The setup of the experiment was no simpler than this:

```
data _null_ ;
  length K D 8 ;
  hxp = 16 ; key = "k" ; data = "d" ;
  dcl hash h (hashexp: hxp) ;
  h.definekey  (key) ;
  h.definedata (key, data) ;
  h.definedone () ;
  k = 1 ; d = 2 ; h.add() ;
run ;
```

And guess what? SAS swallowed it hook, line, and sinker with no slightest sign of protest. But if valued variables work instead of literals, both being SAS expressions, why would not any general SAS expression at all work? As further experimentation has shown, there is no reason why not: Indeed, any general expression works – as long as it resolves to the required value of the required data type. The consequences of this discovery are enormous since it reveals the *true dynamic nature* of the hash object. Because of this property, for example, a hash object itself can be constructed at run time conditionally - for example, according to a control table – and directly in the DATA step where it is used - and without resorting to macros or other methods of assembling SAS code.

Another revelation came from modifying this simple code to answer the question, which first seemed like a stretch: Do all the key and data components for the future hash object instance have to be passed to the DEFINEKEY/DEFINEDATA methods *at once* – or can they be added one at a time *in separate calls*? Separating the DEFINEDATA call above into two calls – one for KEY and the other – for DATA would look like this:

```
data _null_ ;
  length K D 8 ;
  hxp = 16 ; key = "k" ; data = "d" ;
  dcl hash h (hashexp: hxp) ;
  h.definekey  (key) ;
* h.definedata (key, data) ;
  h.definedata (key) ;
  h.definedata (data) ;
  h.definedone () ;
  k = 1 ; d = 2 ; h.add() ;
run ;
```

Testing this step confirmed the hypothesis: Separate DEFINEDATA calls work just as well as a single call for both hash variables. Further tests confirmed the same for DEFINEKEY. From there, it was easy to extrapolate that defining hash variables one at a time can be done by enclosing a call to either method *in a loop*.

As one of the simplest examples using such functionality, suppose that we:

- Have a data set HAVE we want to load into a hash table.
- Want all the variables from HAVE whose names begin with K to become hash's composite key, i.e. define all of them in the key portion.
- Want all the variables whose names begin with D to be defined in the data portion.

If we went the route of using literals – the only one the docs suggest – we would have two ways of doing it:

1. The first, most direct, way would be to find all the required variables and hard-code their quoted, comma-separated names into the DEFINEKEY and DEFINEDATA calls. This would be quite tedious and error prone.
2. The other way would be to compose the required comma-separated sequence using a macro or another technique outside the DATA step in question. It is better than #1 but still underutilizes hash object's dynamic run-time nature.

However, the experimentally discovered functionality discussed above offers a much neater way. The data set HAVE with 3 records (we do not need more for a test) can be emulated, for example, as:

```
data have ;
  array k [1:3] (1 2 3) ;
  array d [1:5] (1 2 3 4 5) ;
  do k1 = 1 to 3 ;
    output ;
  end ;
run ;
```

Now to define the keys and data dynamically and load HAVE into the instance of H, we can code, for instance:

```
   data _null_ ;
     if 0 then set have (keep = k: d:) ; * parameter-type match K: and D: variables;
     dcl hash h (dataset: "have") ;
     length _vn $ 32 ; * receptacle for variable name values ;
     do until (cmiss (_vn)) ;
       call vnext (_vn) ;
       if      _vn =: "k" then h.definekey  (_vn) ;
       else if _vn =: "d" then h.definekey  (_vn) ;
     end ;
     h.definedone () ;
     h.output (dataset: "hash") ; * check hash data content ;
     stop ;
   run ;
```

Above, the VNEXT call routine is used to go over the variables from HAVE in a loop. The IF conditions skip any variables, whose names do not start with "k" or "d". It could be done differently, for example by reading SASHELP.VCOLUMN in a DO loop or looping through arrays defined as _NUMERIC_ and _CHAR_ (do it if you want it as a useful exercise). But no matter what the choice, the crux of the matter is relying on two empirically discovered properties:

- The arguments of the hash methods are general expressions, not just literals.
- The DEFINE methods can be called one key and/or data component at a time.

HOW DO THINGS REALLY WORK?

Experimenting with SAS hash object code sometimes reveals that compared to the docs, in reality:

- Some things, such as the REF method, do not work exactly as described.
- Important caveats of others, such as the DO_OVER method, are not covered, which can lead to a programming trap.
- Solutions offered by SAS Support to resolve some serious problems, such as the "stuck hash iterator" issue, are not quite efficient.

Let us look at the three cases mentioned above separately.

The REF Hash Method

This method, which became available as an upgrade some time after Version 9.0, is quite handy because it avoids unnecessary conditional coding. Namely, given a key-value, the method:

- *Checks* if the key-value already exists in the hash table referenced.
- If it exists, the method does nothing.
- Otherwise, it adds the key-value and the corresponding data-values to the hash table as another item.

Because of this, REF, unlike ADD, is always successful returning a zero execution return code. Describing the way it works, the docs say that the REF method's actions are equivalent to the following with respect to table H:

```
if h.find() ne 0 then h.add() ;
```

Seemingly, code-wise it aligns with the above description. Indeed, if the FIND method returns a non-zero value, the key-value is not in the table, in which case ADD is called to add the item to the table successfully. However, there is a subtlety: The *FIND method not only checks if the key-value is in the table*, but when it is in the table, it *overwrites the host data-values in the PDV* with the data-values from the corresponding hash variables in the table. This is different from the behavior of the CHECK method, which *only checks if the key-value is in the table* – and does nothing else beyond it. So, how does REF behave in reality – like ADD predicated by FIND or ADD predicated by CHECK?

The experiment below tests exactly the case that counts, i.e. *when the key-value is already in the table*. Note that before the REF call, the hash variable in the table D=1, while the corresponding host PDV variable D is set to null on purpose. If after REF is called, the host data-value of D in the PDV remains null, then REF relies internally on CHECK; otherwise, it relies on FIND.

```
data _null_ ;
  dcl hash h() ;
  h.definekey  ("k") ;
  h.definedata ("d") ;
  h.definedone () ;
  k = 1 ; d = 1 ; h.add() ;
  d = . ; * empty host variable ;
  h.ref (key:1, data:2) ;
  put k= d= ;
run ;
```

After this step runs, the log reveals:

```
k=1 d=.
```

Hence, in reality, the logic of the REF method is:

```
if h.check() ne 0 then h.add() ;
```

Interestingly, I reported this discrepancy in the docs to SAS Support 22 years ago, and yet this obvious error in the docs is still there.

The Unstoppable DO_OVER

The DO_OVER method, introduced in SAS 9.2. applies to the situation when a hash table has more than one item keyed by the same key-value. For example, imagine a hash table H with the following content:

```
K  |  D
---+---
6  |  1
6  |  2
6  |  3
6  |  4
6  |  5
6  |  6
```

In other words, it has 6 items with the same key-value 6. The purpose of the DO_OVER method is to, given a key-value, access the same-key group of items with that key-value and process them sequentially - *apparently*, starting with the first item. In the following step, DO_OVER is used to enumerate all the hash items with the key-value = 6 and print the hash values in the log:

```
data _null_ ;
  dcl hash h   (multidata: "y", ordered: "a") ;
  h.definekey  ("k") ;
  h.definedata ("d") ;
  h.definedone () ;
  * load H ;
  K = 6 ;
  do D = 1 to k ;
    h.add() ;
  end ;
  * print values with key-value=6 ;
  do while (h.do_over(key:6) = 0) ;
    put k= d= ;
  end ;
run ;
```

And indeed, DO_OVER duly prints all the requisite values in the log, no issues at this point:

```
K=6 D=1
K=6 D=2
K=6 D=3
K=6 D=4
K=6 D=5
K=6 D=6
```

I welcomed the method enthusiastically upon its arrival because before its advent, if I wanted to do the same exact thing, I would have to code something like this instead of the simple DO WHILE loop above:

```
do rc = h.find(key:6) by 0 until (h.find_next() ne 0) ;
  put k= d= ;
```

```
end ;
```

No big deal, really – it certainly works and lists the same values as the loop with DO_OVER – but the code with DO_OVER looks and feels much simpler. Now imagine that we wanted to do the following instead of simple enumeration:

- If within the item group with a given key-value (in this case, K = 6) we find D = 3, exit the loop and flag the group as needed to be printed by assigning print = 1.
- If print = 1, go over the whole item group again and print the entire group (just like in the example above).
- Otherwise, do nothing.

Accordingly, I would code the following, fully expecting that the step will print the same 6 lines – i.e. the whole item group - as above:

```
data _null_ ;
  dcl hash h   (multidata: "y", ordered: "a") ;
  h.definekey  ("k") ;
  h.definedata ("d") ;
  h.definedone () ;
  K = 6 ;
  do D = 1 to k ;
    h.add() ;
  end ;
  do while (h.do_over(key:6) = 0) ;
    if d = 3 then do ;
      print = 1 ;
      leave ;
    end ;
  end ;
  if print then do while (h.do_over(key:6) = 0) ;
    put K= D= ;
  end ;
run ;
```

What it printed instead, however, was only this:

```
K=6 D=4
K=6 D=5
K=6 D=6
```

Huh? What gives? After some more tests and respective cogitation, it dawned on me: If DO_OVER started enumerating a same-key item group, it cannot be stopped and told to start from the beginning of the group until it has gone through all the items. If its calls are stopped before the DO loop is over, the next DO_OVER call will pick up from the item to be enumerated next. In this case, the first DO loop was stopped at D=3 by LEAVE because from the standpoint of efficiency there was no reason to continue once print = 1 flag was set. But when the next DO loop was launched, the DO_OVER call in its body began from the item next to enumerate after the last call in the previous DO loop.

None of the above is mentioned or explained in the docs. Furthermore, the name of the method, DO_OVER feels deceptive because SAS has a special loop for implicitly subscripted arrays DO OVER – which, unlike the DO_OVER method, always starts at item 1, regardless of which item the array loop was dwelling before.

So, the solution is to deep-six DO_OVER and use the DO loop with the FIND_NEXT method because the initial FIND always restarts it from the first item of the same-key item group:

```
   data _null_ ;
     dcl hash h   (multidata: "y", ordered: "a") ;
     h.definekey  ("k") ;
     h.definedata ("d") ;
     h.definedone () ;
     K = 6 ;
     do D = 1 to k ;
       h.add() ;
     end ;
     do rc = h.find(key:6) by 0 until (h.find_next() ne 0) ;
       if d = 3 then do ;
         print = 1 ;
         leave ;
       end ;
     end ;
     if print then
     do rc = h.find(key:6) by 0 until (h.find_next() ne 0) ;
       put K= D= ;
     end ;
   run ;
```

This time, the outcome was as expected:

```
K=6 D=1
K=6 D=2
K=6 D=3
K=6 D=4
K=6 D=5
K=6 D=6
```

The code element BY 0 might need your attention if you have never used the trick. If it were not there, RC would have only the <from> DO loop index value without the <to> value, and the loop would stop after the first iteration. BY 0 tells the loop to keep iterating endlessly until it is stopped by another mechanism – in this case, h.find_next()=0 or LEAVE.

The Stuck Hash Iterator Problem

Let us consider the following experimental setup:

```
data _null_ ;
  dcl hash h() ;
  h.definekey  ("k") ;
  h.definedone () ;
  do K = 1 to 7 ;
    put k  ;
    h.add() ;
  end ;
  dcl hiter hi ("h") ;
  do while (hi.next() = 0) ;
    if k = 3 then leave ;
  end ;
  rc = h.remove (key:3) ;
```

```
* rc = h.clear() ;
run ;
```

Above, the hash iterator HI, linked to the instance of hash H, is used to enumerate the hash table, i.e. to access its items serially starting from the first item. The content of the hash table H above, by design, is:

```
K
--
1
2
3
4
5
6
7
```

The code tells the program to stop enumerating at K=3 and leave the DO loop. Then the program attempts to remove the hash item with K=3, and this results in the following message in the SAS log:

```
ERROR: Cannot remove record from a hash object which has been locked by an iterator
at line 13 column 8.
ERROR: DATA STEP Component Object failure.  Aborted during the EXECUTION phase.
```

If, instead of REMOVE, CLEAR is called, the error message is the same, just on 14 instead of 13. That should be no surprise since CLEAR attempts to *remove all items* from the table, and if any item is locked by the iterator, CLEAR will fail. To make the discussion simpler, let us call the imaginary thing pointing to the hash table location on which the iterator currently dwells, an *iterator pointer*. If the iterator is not dwelling on any table item, let us say that the *pointer is out of the table*, in which state the *table is not locked*. Au contraire, if the pointer dwells on any item, this item and maybe some others may be locked by the iterator, i.e. they cannot be removed from the table.

After having experimented with this problem in several ways, I came to the following conclusions:

- If the iterator pointer dwells on a hash item, it is locked – it cannot be removed from the table.
- Every item sharing the key-value with the item, on which the pointer dwells, is also locked and cannot be removed. This, obviously, can happens only in a table with duplicate-key items.
- If the iterator dwells anywhere inside the table, the CLEAR method will fail because at least one of the items it attempts to remove is locked.

Apparently, to be able to unlock an item with a specific key-value, *we need to move the pointer away* from that item - and any item with the same key – either to an item with a different key-value or out of the table. If we need to unlock every hash item to make the CLEAR method successful, we need to move the iterator pointer out of the table. But it raised the question: How would we do that?

At the time I first ran into this issue, nobody had reported it yet – not surprisingly, given that then practically nobody in the user community had SAS 9.0 at one's fingertips. I did report it, and eventually, the issue received a Technical Note. It offered the following "empty loop" solution:

```
do while (hi.next() = 0) ;
end ;
```

It did work – but instead of simply accepting it, it was more instructive to understand why. Let's see here: After the hi.next() call enumerated the last hash item, hi.next() was called again. Since the table had no more items, the call failed with a non-zero return code, or otherwise the WHILE loop would not have

stopped. But from the experimental fact that the table was no longer locked, I had to unescapably conclude that the failed call kicked the pointer out of the table.

However, any performance-inclined programmer cannot help but wonder: It the table contains, say, 2 million items (which is not too many nowadays) and the iterator locks some random item, this technique of pushing the iterator pointer out of the table would scan on average 1 million items just to remove the iterator lock and doing nothing else useful. This looked wasteful, so was there a better way? To answer this question, I had to ask if there were any hash methods that move the iterator pointer *directly to the beginning or end* of the table from the item it dwells on *without having to scan every item* in between.

Fortunately, the answer was: Yes: there are the methods FIRST and LAST that do exactly that. So, I could use either one to move the pointer directly to the first or last item, respectively, and thus eschew the unnecessary scanning. However, that still would not move the pointer out of the table yet - the pointer, whilst one step away from being *outside* the table, would still be *inside* the table. How can we push it out? Well, we know that NEXT eventually pushes the pointer outside the table because its last call tries to enumerate a non-existing item. We can use a similar tactic: Once the pointer is at the beginning or the end after calling FIRST or LAST, *what if* we then call PREV or NEXT, respectively? In principle, it should work similarly to the last call in the WHILE loop above. Expectedly, the call would fail in terms of finding the previous or next hash item – but, while failing, it would move the iterator pointer outside the table! Or would it not? Armed with this hypothesis, I ran the following experiment:

```
data _null_ ;
  dcl hash h() ;
  h.definekey  ("k") ;
  h.definedone () ;
  do K = 1 to 7 ;
    put k  ;
    h.add() ;
  end ;
  dcl hiter hi ("h") ;
  do while (hi.next() = 0) ;
    if k = 3 then leave ;
  end ;
  hi.first() ;  rc = hi.prev() ;
 *hi.last()  ; *rc = hi.next() ;
  rc = h.remove (key:3) ;
  h.clear() ;
run ;
```

As it turned out, it worked just fine, returning no log errors. I must confess that it was not on the first try because originally, I coded the hi.prev() call *unassigned*: (note the absent rc =):

```
    hi.prev() ;
```

That is, it was called without assigning its return code to the RC variable. That resulted in the error message: `ERROR: Key not found.` At first, I was puzzled but then realized that this was a proper reaction to hi.prev() failing to find the next item, since any unassigned call issues and error message if it fails. Changing that to the assigned call with made the error message go away. Using LAST/NEXT (commented out above) instead of FIRST/PREV worked without fail, too. The only question left to be elucidated was how much run time we would save by skipping directly to the beginning/end of the table rather than by stepping to the endpoints one item at a time. It is not difficult to find out using this experimental setup:

```
data _null_ ;
   dcl hash h() ;
```

```
  h.definekey  ("k") ;
  h.definedone () ;
  do K = 1 to 2e6 ; * load 2,000,000 items ;
    h.add() ;
  end ;
  dcl hiter hi ("h") ;

  hi.setcur (key: 1e6) ; * move HI pointer to middle ;
* SAS Support suggestion ;
  t = time() ;
  do while (hi.next() = 0) ;
  end ;
  t1 = time() - t ;

  hi.setcur (key: 1e6) ; * move HI pointer to middle ;
* LAST/NEXT or FIRST/PREV technique ;
  t = time() ;
  hi.last() ; rc = hi.next() ;
  t2 = time() - t ;
  put t1= / t2= ;
run ;
```

This experiment is set up to do the following:

- Load the hash table H with 2 million items.
- Use the SETCUR method to place the iterator pointer at the middle of the table.
- Use the technique suggested by SAS Support to kick the iterator pointer out of the table and compute the elapsed time.
- Use the SETCUR method to place the iterator pointer at the middle of the table.
- Use the technique based on the LAST/NEXT combo to move the pointer out of the table and compute the elapsed time.

Running this step results in this log note:

```
t1=0.0480000973
t2=0
```

As you can see, the run time for the LAST/NEXT technique is so infinitesimally small that it is not even measurable; even formatting it with HEX16. returns all zeroes.

CONCLUSION

Experimental programming is a powerful tool for exploring the SAS hash object hidden functionality far beyond what is found between the covers of the SAS manual.

CONTACT INFORMATION

Your comments and questions are valued and encouraged. Contact the author at:

Paul M. Dorfman
sashole@gmail.com

SAS and all other SAS Institute Inc. product or service names are registered trademarks or trademarks of SAS Institute Inc. in the USA and other countries. ® indicates USA registration.
Other brand and product names are trademarks of their respective companies.